

# กลุ่ม 15 — Generics Type System

1. นายณัฐวุฒิ โตเมือง (670710623) — Concept + Short Code Illustration
2. นางสาวณัฐสุดา ลานทวน (670710624) — Detailed Code + Live Demo
3. นางสาวณัฐสุรางค์ ชาทิทองคำ (670710625) — Rust vs Other Language + PPL Analysis
4. นายธนเทพ นาสวน (670710626) — Exercises + Common Mistakes + Challenge

# Agenda — โครงสร้างการนำเสนอ (รวม ~20 นาที)

แต่ละคนนำเสนอประมาณ 5 นาที ต่อเนื่องกันตามลำดับ

1

**Member 1 : นายณัฐวุฒิ โตเมือง**

**Concept + Short Code  
Illustration**

แนวคิดหลักของหัวข้อ พร้อมโค้ดตัวอย่าง  
สั้น ๆ

~5 นาที

2

**Member 2 : นางสาวณัฐสุดา ลานตวน**

**Detailed Code + Live Demo**

อธิบายโค้ดเชิงลึกและสาธิตการทำงานจริง

~5 นาที

3

**Member 3 : นางสาวณัฐสุรางค์ ชาทิทองคำ**

**Rust vs Other Language + PPL  
Analysis**

เปรียบเทียบภาษา และวิเคราะห์เชิง  
Programming Language

~5 นาที

4

**Member 4: นายธนเทพ นาสวน**

**Exercises + Common Mistakes +  
Challenge**

Mistakes 1 & 2 (Nominal Contracts &  
Vtable) + Exercises 1 & 2 (Cache &  
Notifier) + Challenge Trade-offs

~5 นาที

# Generics & Type System คืออะไร?

Generics คือการใช้ Type Parameter เช่น T  
เพื่อเขียน Function หรือ Struct  
ให้สามารถใช้กับข้อมูลหลายชนิดได้

## ทำไม Rust จึงใช้ Generics?

- ลดการเขียนโค้ดซ้ำ (Code Reuse)
- ใช้ Abstraction เดียวกับหลาย Concrete Types
- ยังคงให้ Compiler ตรวจสอบชนิดข้อมูลตั้งแต่ Compile Time

## แนวคิดสำคัญ

**“เขียน Logic ครั้งเดียว และนำไปใช้กับหลาย Type  
โดยยังคง Type Safety”**

- Parametric Polymorphism

Generics ทำให้เราสามารถเขียน Logic ที่ไม่ได้ผูกกับชนิดข้อมูลใดชนิด  
หนึ่งตั้งแต่ต้น แต่ระบุ Type จริงเมื่อมีการนำไปใช้งาน

```
fn identity<T>(value: T) -> T {
    value
}

fn main() {
    println!("{}", identity(42));
    println!("{}", identity("Rust"));
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
42
Rust
```

# Generic Function & Type Parameter

- Generic Function คือ Function ที่ใช้ Type Parameter

แทน Concrete Type

## Syntax ที่ควรรู้

- <T>** → ประกาศ Type Parameter
- value: T** → Parameter มี Type เป็น **T**
- > T** → Return Type เป็น **T**

## หลักการทำงาน

เมื่อเรียกใช้ Function → Type T จะสอดคล้องกับ Type ของค่าที่ใช้งาน

ตัวอย่าง

identity(42) → T = i32

identity("Rust") → T = &str

***Function เดียว → รองรับหลาย Type***

```
fn identity<T>(value: T) -> T {
    value
}

fn main() {
    let number = identity(42);
    let text = identity("Rust");

    println!("number = {}", number);
    println!("text = {}", text);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
number = 42
text = Rust
```

# Generic Struct & Monomorphization

```
#[derive(Debug)]
struct Point<T> {
    x: T,
    y: T,
}

fn main() {
    let int_point = Point { x: 5, y: 10 };
    let float_point = Point { x: 1.5, y: 4.0 };

    println!("{int_point:?}");
    println!("{float_point:?}");
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Point { x: 5, y: 10 }
Point { x: 1.5, y: 4.0 }
```

## Generic Struct

- Struct ที่ใช้ Type Parameter เพื่อรองรับข้อมูลหลาย Type

## หลักการ

- **Point<T>** → Generic Struct
- **Point<i32>** → ใช้กับจำนวนเต็ม
- **Point<f64>** → ใช้กับเลขทศนิยม

## Type Inference

Rust สามารถอนุมาน Concrete Type จากค่าที่กำหนดให้

## Monomorphization

- Generic Code ถูกนำไปสร้างรูปแบบที่สอดคล้องกับ Concrete Type ที่ถูกใช้งานในขั้นตอน Compile

**Generic Code เดียว → รองรับหลาย Concrete Types**

# Detailed Code + Live Demo 1

```
#[derive(Debug)]
struct KeyValue<K, V> {key: K,value: V,}

impl<K, V> KeyValue<K, V> {
    fn new(key: K, value: V) -> Self {KeyValue {
        key, value }
    }
    fn unpack(self) -> (K, V) {
        (self.key, self.value)
    }
}

impl<K> KeyValue<K, f64> {
    fn is_positive_metric(&self) -> bool {
        self.value > 0.0}
}
```

## Demo Checklist

- ❑ `#[derive(Debug)]` → ใช้ `{:?}` แสดงข้อมูล Struct
- ❑ Generic Struct: `KeyValue<K, V>` รองรับข้อมูลได้หลาย Type
- ❑ Generic Implementation
  - `new()` สร้าง `KeyValue`
  - `unpack()` แยก `key` และ `value`
- ❑ Specialized Implementation
  - `is_positive_metric()` ใช้เฉพาะ `V = f64`
  - ตรวจสอบว่า `value > 0.0`
- ❑ สิ่งที่ต้องระวังระหว่าง Demo
  - `is_positive_metric()` ใช้ได้เฉพาะ `f64`

## Detailed Code + Live Demo 1 (ต่อ)

```
fn find_min<T: PartialOrd + Copy>(list: &[T]) -> Option<T>{
    if list.is_empty() {
        return None;
    }
    let mut min = list[0];
    for &item in list.iter().skip(1) {
        if item < min {
            min = item;
        }
    }
    Some(min)
}
```

### Demo Checklist

- Generic Function: find\_min()
  - หา Minimum ของข้อมูลแต่ละ Type
  - PartialOrd ใช้เปรียบเทียบค่า
  - Copy คัดลอกค่า
- สิ่งที่ต้องระวังระหว่าง Demo
  - find\_min() ต้องมี PartialOrd + Copy

# Detailed Code + Live Demo 1 (ต่อ)

```
fn main() {
    let sensor = KeyValue::new("Temperature", 36.6);
    println!("Sensor Record: {:?}", sensor);
    println!("Is positive metric: {}", sensor.is_positive_metric());

    let user_flag = KeyValue::new(1001, true);
    println!("User Flag: {:?}", user_flag);
    let (uid, flag) = user_flag.unpack();
    println!("Unpacked -> UID: {}, Status: {}", uid, flag);

    let scores = [85, 92, 78, 90];
    let temps = [36.6, 37.2, 35.8, 38.0];

    println!("Min Score: {:?}", find_min(&scores));
    println!("Min Temp : {:?}", find_min(&temps));
}
```

## Demo Checklist

### ☐ Type Inference

- Compiler ตรวจสอบ Type ให้อัตโนมัติ
- scores คือ i32
- temps คือ f64

# Detailed Code + Live Demo 2

```
use std::any::type_name;
use std::mem::size_of;

fn inspect_execution<T: std::fmt::Display>(val: T) {
    println!(
        "Type: {:<5} | Value: {:<5} | Function Address: {:p}",
        type_name::<T>(),
        val,
        inspect_execution::<T> as *const ()
    );
}

struct Storage<T> { data: T, }

impl<T> Storage<T> {
    fn get(&self) -> &T {
        &self.data
    }
    fn byte_size(&self) -> usize {
        size_of::<T>()
    }
}
```

## Demo Checklist

- ❑ Trait Bounds (<T: Display>)
  - บังคับให้ Type T ต้องพิมพ์ออกจอได้ (Compile-time guarantee)
- ❑ Inspecting Function Address
  - inspect\_execution::<T> as \*const ()
  - ดึง Address จริงของโค้ดใน Memory เพื่อดูการสร้างฟังก์ชัน
- ❑ Zero-Cost Generic Struct
  - ใช้ size\_of::<T>() เพื่อดูขนาด Memory
  - พิสูจน์ว่า Generic เพิ่ม Overhead หรือไม่

# Detailed Code + Live Demo 2 (ต่อ)

```
fn main() {
    println!("=== 1. Function Pointer Monomorphization ===");
    inspect_execution(100_i32);
    inspect_execution(200_i32);

    inspect_execution(3.14_f64);
    inspect_execution(true);

    println!("\n=== 2. Struct Memory Layout ===");
    let int_store = Storage { data: 42_i32 };
    let byte_store = Storage { data: 42_u8 };
    let float_store = Storage { data: 42.0_f64 };
    println!(
        "Storage<i32> holds value: {:<4} | consumes: {} bytes",
        int_store.get(), int_store.byte_size()
    );
    println!(
        "Storage<u8> holds value: {:<4} | consumes: {} bytes",
        byte_store.get(), byte_store.byte_size()
    );
    println!(
        "Storage<f64> holds value: {:<4} | consumes: {} bytes",
        float_store.get(), float_store.byte_size()
    );
}
```

## Demo Checklist

- ❑ Same Type → Reuse Code
  - i32 (100) และ i32 (200) ได้ Function Address เดียวกัน
  - คอมไพเลอร์ใช้โค้ดชุดเดิม ไม่สร้างซ้ำ
- ❑ Different Types → Split Code )
  - f64 และ bool ได้ Function Address ต่างกัน
  - Rust แอบสร้างฟังก์ชันเฉพาะของแต่ละ Type ให้ในระดับ Binary
- ❑ Struct Memory Layout
  - u8 = 1 byte | i32 = 4 bytes | f64 = 8 bytes
  - ขนาดเท่า Type ดั้งเดิมเป๊ะ (Zero-cost Abstraction)
- ❑ สิ่งที่ต้องระวังระหว่าง Demo
  - Address จะเปลี่ยนทุกครั้งที่รันใหม่
  - Code Bloat ยิ่งใช้ Type หลากหลาย ขนาดไฟล์ Binary จะยิ่งบวมขึ้น
  - Compile Time คอมไพล์ช้าลงเพราะคอมไพเลอร์ต้องแตกโค้ดและทำ Optimization ซ้ำๆ

# Rust vs Java

| Aspect               | Rust                                                                                                    | Java                                                                                                                                              |
|----------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Syntax               | ใช้ <T> ระบุ Type Parameter และใช้ Trait Bound กำหนดข้อจำกัด                                            | ใช้ <T> เช่นกัน แต่กำหนดข้อจำกัดผ่าน extends                                                                                                      |
| Semantics / Behavior | ใช้ Monomorphization สร้าง Code สำหรับแต่ละ Type ที่ใช้ และตรวจสอบความถูกต้องตั้งแต่ Compile Time       | ใช้ Type Erasure ทำให้ Generic Type ส่วนใหญ่ไม่คงอยู่ตอน Runtime และ Compiler อาจแทรก Cast ตามบริบท โดย Primitive Type อาจต้องใช้ Boxing/Unboxing |
| Type System          | Static Type + Trait System โดยตรวจสอบ Trait ตั้งแต่ก่อน Compile จริง                                    | Static Type และตรวจสอบ Bound ตอน Compile Time แต่ Generic Type ถูก Erasure                                                                        |
| Memory Management    | ใช้ Ownership + Borrowing จัดการ Memory โดยไม่ต้องมี Garbage Collector และควบคุมอายุ Resource ได้ชัดเจน | ใช้ Garbage Collector จัดการ Memory อัตโนมัติ แต่เวลาคืน Memory ไม่แน่นอน                                                                         |
| Safety               | ลดปัญหา Memory Safety ตั้งแต่ Compile Time                                                              | Reference สามารถเป็น null จึงอาจเกิด NullPointerException ตอน Runtime                                                                             |

# Rust vs Python

| Aspect               | Rust                                                                               | Python                                                                              |
|----------------------|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Syntax               | ใช้ <T> และ Trait Bound กำหนดข้อจำกัดของ Type                                      | ใช้ TypeVar และ Generic ผ่าน typing เป็น Type Hint                                  |
| Semantics / Behavior | Compiler ตรวจสอบและ Monomorphize ตอน Compile Time ได้ Native Code                  | Python Runtime ไม่บังคับ Type Annotation แต่ใช้เครื่องมือช่วยตรวจ Type ก่อน Runtime |
| Type System          | Static Type + Trait System ตรวจสอบ Type และ Trait ตั้งแต่ Compile Time             | Dynamic Type + Duck Typing ตรวจสอบพฤติกรรมSSM/Type ของ Object ขณะ Runtime           |
| Memory Management    | ใช้ Ownership + Borrowing โดยไม่ต้องมี Garbage Collector                           | ใช้ Reference Counting + Garbage Collector จัดการ Memory อัตโนมัติ                  |
| Safety               | Compiler ตรวจสอบ Type และ Ownership ก่อนทำงาน ช่วยลดข้อผิดพลาดตั้งแต่ Compile Time | ข้อผิดพลาดด้าน Type มักตรวจพบตอน Runtime และอาจเกิด TypeError                       |

# Rust vs C++

| Aspect               | Rust                                                                                      | C++                                                                                               |
|----------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Syntax               | ใช้ <T> และ Trait Bound สำหรับ Generic โดยตรง                                             | ใช้ Template เช่น template <typename T> และสามารถใช้ Concepts ใน C++20 เพื่อกำหนดข้อจำกัดของ Type |
| Semantics / Behavior | Compiler ตรวจสอบและ Monomorphize ตอน Compile Time สร้างโค้ดเฉพาะแต่ละ Type                | Compiler ทำ Template Instantiation ตอน Compile Time เพื่อสร้างโค้ดตาม Type ที่ใช้งาน              |
| Type System          | Static Type + Trait System บังคับให้ Type มีพฤติกรรมตาม Trait ที่กำหนด                    | Static Type System และใช้ Template กำหนดข้อจำกัดของ Type ได้ ตรวจสอบข้อกำหนดตั้งแต่ Compile Time  |
| Memory Management    | ใช้ Ownership + Borrowing จัดการ Memory โดยไม่ต้องมี Garbage Collector                    | จัดการ Memory ได้ทั้งแบบ Manual และ RAII โดยไม่มี Garbage Collector แบบ Java                      |
| Safety               | Compiler ตรวจสอบ Type และ Ownership ช่วยป้องกัน Memory Bug หลายประเภทตั้งแต่ Compile Time | ไม่มีระบบ Memory Safety แบบ Rust โดยตรง จึงยังสามารถเกิดปัญหาได้                                  |

# PPL Perspective — มุมมองเชิง Programming Language

## Syntax

- ใช้ `<T>` สำหรับประกาศ Generic Type Parameter
- ใช้ Trait Bound เช่น `T: PartialOrd` เพื่อกำหนดข้อจำกัดของ Type
- รองรับ Parametric Polymorphism

## Semantics

- Generic Function ทำงานกับหลาย Type
- Trait Bound กำหนดความสามารถที่ Type ต้องมี
- เป็นการใช้ Parametric + Bounded Polymorphism

## Type System

- ใช้ Generics + Traits เพื่อรองรับหลาย Type
- Compiler ตรวจสอบ Type และ Trait Bound ตอน Compile Time ช่วยเพิ่ม Type Safety

## Memory / Resource

- ทำงานร่วมกับ Ownership, Borrowing และ Lifetime
- Compiler ตรวจสอบ Memory Safety ตอน Compile Time
- ใช้ Monomorphization สร้าง Code ตาม Concrete Type

## Abstraction

- ใช้ Trait กำหนดพฤติกรรมของ Type
- ใช้ Composition + Trait แทน Class Inheritance
- Generics ช่วยสร้าง Abstraction ที่นำกลับมาใช้ได้

## Why Rust?

- ตรวจสอบ Type และ Memory ตั้งแต่ Compile Time
- Trait Bound กำหนดข้อจำกัดของ Type ชัดเจน
- Monomorphization ช่วยลด Runtime Overhead

## Common Mistake 1 — Missing Trait Bounds

แสดงข้อจำกัดของการคำนวณทางคณิตศาสตร์บนชนิดข้อมูลปลายเปิด และการทำงานของกลไก Nominal Contract Enforcement ที่คอมไพเลอร์บังคับตรวจสอบสัญญาล่วงหน้าตั้งแต่จุดประกาศฟังก์ชัน เพื่อขจัดข้อผิดพลาดก่อนถึงขั้นตอนรันไทม์

❌ **โค้ดที่ผิด: เรียกใช้ Operation บน Generic โดยไร้ Trait Bound**

```
// ❌ ผิด: พยายามบวก Type T โดยไม่มีข้อดกลง (Trait Bound)
fn calculate_sum_incorrect<T>(a: T, b: T) -> T {
    a + b // ERROR[E0369]: cannot add `T` to `T`
          // `T` might need a bound for `std::ops::Add`
}
```

✅ **โค้ดที่ถูกต้อง: กำหนดข้อกำหนดคุณสมบัติผ่าน where clause**

```
use std::ops::Add;

// ✅ ถูก: กำหนด Trait Bound ว่า T ต้อง Implement std::ops::Add
pub fn calculate_sum_correct<T>(a: T, b: T) -> T
where
    T: Add<Output = T>,
{
    a + b
}

// ใช้งานได้ทั้งจำนวนเต็มและทศนิยม (Type-safe & Zero-cost)
let sum_int = calculate_sum_correct(10, 25); // 35 (i32)
let sum_float = calculate_sum_correct(12.5, 7.5); // 20.0 (f64)
```

### ทำไม Rust ถึงออกแบบแบบนี้?

- Nominal Contract Enforcement at Definition Site:** Rust บังคับตรวจสอบสัญญา ณ จุดประกาศฟังก์ชันทันที ที่ต่างจาก Duck Typing ใน C++ Templates ที่ยอมให้ผ่านเสี่ยงไปเจอข้อผิดพลาดที่ Call Site
- Coherence & Predictability:** การระบุ Trait Bounds ชัดเจนทำให้ Compiler รับประกัน Type-Safety ได้ 100% ตั้งแต่ตอนคอมไพล์ โดยไม่ต้องเห็นประเภทข้อมูลจริงล่วงหน้า

## Common Mistake 2 — Object Safety Violations

แสดงเงื่อนไขความปลอดภัยของวัตถุ (Object Safety) ในการแปลง Trait เป็น Trait Object ซึ่งถูกจำกัดด้วยสถาปัตยกรรมของตาราง Vtable ในหน่วยความจำที่ต้องทราบขนาดและตำแหน่งพอยน์เตอร์ฟังก์ชันที่แน่นอนตั้งแต่คอมไพล์

### ❌ โค้ดที่ผิด: สร้าง dyn Trait ที่มี Generic Method หรือคืนค่า Self

```
pub trait UnsafeObjectTrait {
    // ผิดกฎข้อ 1: Method มี Generic Parameter เป็นของตัวเอง
    // คอมไพเลอร์ไม่สามารถสร้าง Vtable ได้ เพราะไม่รู้ว่าจะมี Type U กี่ชนิด
    fn process<U>(&self, data: U);

    // ผิดกฎข้อ 2: Method คืนค่าเป็น `Self`
    // ที่ Runtime หลังจาก Type Erasure แล้ว ระบบไม่ทราบว่าขนาดของ Self คือกี่ไบต์
    fn clone_self(&self) -> Self;
}

// จะเกิด Error ทันทีเมื่อพยายามสร้าง Trait Object:
let object: Box<dyn UnsafeObjectTrait>; // ERROR: the trait cannot be made
into an object
```

### ✅ โค้ดที่ถูกต้อง: ออกแบบ Trait ให้สอดคล้องกับ Object Safety

```
/// Trait ที่ Object Safe:
/// 1. ไม่มี Generic Methods ที่ต้องสร้าง Vtable ไม่สิ้นสุด
/// 2. ไม่มีการคืนค่า `Self` แบบคลุมเครือ
pub trait SafeLogger {
    fn log(&self, message: &str);
    fn level(&self) -> &'static str;
}

// ใช้งานผ่าน Trait Object ใน Vec ได้อย่างปลอดภัย 100%
let loggers: Vec<Box<dyn SafeLogger>> = vec![
    Box::new(ConsoleLogger),
    Box::new(FileLogger {
        file_name: "app.log".to_string(),
    }),
];
```



### กฎ Object Safety และสถาปัตยกรรมหน่วยความจำ (Memory Layout)

- Vtable Size Must Be Fixed:** ตาราง Virtual Method Table (Vtable) ต้องมีขนาดตายตัวในคอมไพล์ไทม์ หาก Method มี Generic ย่อย จะทำให้เกิดฟังก์ชันได้ไม่จำกัดจำนวนจนไม่สามารถจองขนาด Pointer ใน Vtable ได้
- ข้อจำกัดด้านขนาด (Self: Sized):** dyn Trait ไม่ทราบขนาดหน่วยความจำที่แน่นอน (Unsize / ?Sized) จึงห้าม Method คืนค่า Self โดยตรง เว้นแต่จะห่อด้วย Pointer (เช่น Box หรือ &Self)

## Exercise 1 — Generic In-Memory Cache

การออกแบบโครงสร้างข้อมูลแบบนำกลับมาใช้ซ้ำ (Reusable Generic Data Structure) ร่วมกับเงื่อนไข Trait Bounds, การบริหารหน่วยความจำด้วยนโยบาย FIFO Eviction เมื่อเต็มความจุ และการรับประกันความปลอดภัยจากค่าว่าง (Null-Safety) 100% ด้วยชนิดข้อมูล Option

```
// 1. กำหนด Trait Bounds: K ต้องเทียบ == ได้, V ต้อง Clone สำเนาได้
pub struct SimpleCache<K, V> {
    entries: Vec<CacheEntry<K, V>>, // Private (Encapsulation)
    capacity: usize,
}

impl<K, V> SimpleCache<K, V>
where
    K: PartialEq + Display,
    V: Clone + Display,
{
    /// บันทึกข้อมูล: หากเต็มความจุ เตะตัวแรกทิ้ง (FIFO Eviction)
    pub fn put(&mut self, key: K, value: V) {
        for entry in &mut self.entries {
            if entry.key == key {
                entry.value = value;
                return;
            }
        }
        if self.entries.len() >= self.capacity {
            self.entries.remove(0); // ⚡ FIFO Eviction Policy
        }
        self.entries.push(CacheEntry { key, value, access_count: 1 });
    }

    /// ดึงข้อมูล: ปลอดภัยจาก Null Pointer 100% ด้วย Option<V>
    pub fn get(&mut self, key: &K) -> Option<V> {
        for entry in &mut self.entries {
            if &entry.key == key {
                entry.access_count += 1;
                return Some(entry.value.clone());
            }
        }
        None // 💡 ไม่พบข้อมูล ส่งกลับ None อย่างปลอดภัย
    }
}

// ทดสอบความจุ 3 ชิ้น (Capacity = 3)
let mut cache = SimpleCache::new(3);
cache.put("user_101".to_string(), 95);
cache.put("user_102".to_string(), 80);
cache.put("user_103".to_string(), 88);

// ผ่าขั้นที่ 4 เข้าไป -> เตะ "user_101" ทิ้งทันที!
cache.put("user_104".to_string(), 92);

assert_eq!(cache.get(&"user_101".to_string()), None); // 🗑️ ถูก Evict ออกไปแล้ว
assert_eq!(cache.get(&"user_104".to_string()), Some(92)); // ✅ เก็บข้อมูลใหม่ สำเร็จ
```

### 💡 คุณลักษณะเชิง PPL (Type-Safe Guarantees)

- Parametric Polymorphism & Trait Bounds:** โครงสร้างข้อมูลเดียวรองรับทุก Type (<K, V>) โดยตีกรอบสัญญาความสามารถ K: PartialEq (เทียบ == หาของ), V: Clone (โคลนข้อมูลส่งกลับโดยไม่แย่ง Ownership จาก Cache)
- 100% Null Safety via Option<V>:** ใช้ Some(value) และ None แทนค่า null ทำให้ตัดปัญหา NullPointerException (The Billion-Dollar Mistake) ตั้งแต่ระดับ Type System
- Encapsulation & Information Hiding:** ซ่อนฟิลด์ entries และ capacity ให้เป็น Private (ไม่มี pub) เพื่อป้องกันไม่ให้โค้ดภายนอกแอบยัดข้อมูลโดยข้ามนโยบาย FIFO
- Resource Management:** มีการคุมขนาดหน่วยความจำคงที่ด้วย capacity และระบบ FIFO Eviction (remove(0)) ป้องกันปัญหา Memory Leak หรือ Out of Memory

## Exercise 2 — Polymorphic Notification Engine

การเปรียบเทียบสถาปัตยกรรมการส่งข้อมูลระหว่าง Static Dispatch ที่ทำงานรวดเร็วระดับ Zero-Cost สำหรับงานด่วนฉุกเฉิน กับ Dynamic Dispatch ที่ทำงานผ่านตัวชี้คู่ (Fat Pointer ขนาด 16 ไบต์) เพื่อจัดเก็บชนิดข้อมูลที่หลากหลายไว้ในคอลเลกชันเดียวกัน

```
pub trait Notifier {
    fn channel_name(&self) -> &'static str;
    fn send(&self, recipient: &str, message: &str) -> Result<(), String>;
}

// ⚡ 1. Static Dispatch: Monomorphization (Early Binding, เร็วสูงสุดระดับ Zero-Cost)
pub fn send_urgent_static<N: Notifier>(notifier: &N, recipient: &str, alert: &str) {
    let _ = notifier.send(recipient, alert); // คอมไพเลอร์ Direct Call ตรงๆ
}

// 🌐 2. Dynamic Dispatch: รวมหลาย Channel ต่างชนิดไว้ใน Vec เดียว (Heterogeneous)
pub struct NotificationBroadcaster {
    channels: Vec<Box<dyn Notifier>>, // 📦 Fat Pointer ขนาด 16 ไบต์ (8B Heap + 8B Vtable)
}

impl NotificationBroadcaster {
    pub fn broadcast(&self, recipient: &str, message: &str) {
        for channel in &self.channels {
            let _ = channel.send(recipient, message); // เรียกค้นหาฟังก์ชันผ่าน Vtable
        }
    }
}
```

```
// 1. Static Dispatch: ส่งเดี่ยวแบบเร็วพิเศษ
send_urgent_static(&email, "admin@su.ac.th", "High CPU Load!");

// 2. Dynamic Dispatch: รวม 3 ช่องทางต่าง Type ลงใน Collection เดียวกันได้!
let mut broadcaster = NotificationBroadcaster::new();
broadcaster.register_channel(Box::new(email));
broadcaster.register_channel(Box::new(SmsNotifier { country_code: "66".to_string() }));
broadcaster.register_channel(Box::new(DiscordWebhookNotifier { bot_name: "AI".to_string() }));

broadcaster.broadcast("student@su.ac.th", "Project Deadline: 4 Oct!");
```



### Fat Pointer Architecture & Error Handling

- 1.Static Dispatch (Early Binding):** ใช้ <N: Notifier> ทำให้เกิด Monomorphization สร้างฟังก์ชันตรงตาม Type คอมไพเลอร์ Inlined โค้ดได้ ไม่ต้องพึ่งพาตารางฟังก์ชัน (Zero-Cost Abstraction) แต่ข้อจำกัดคือเก็บรวมหลาย Type ไม่ได้
- 2.Dynamic Dispatch (Late Binding):** ใช้ Box<dyn Notifier> ซึ่งทำงานผ่าน Fat Pointer ขนาด 16 ไบต์ (8 ไบต์ชี้ข้อมูลจริงบน Heap + 8 ไบต์ชี้ตาราง Vtable) ทำให้สร้าง Heterogeneous Collection (รวม Email, SMS, Discord ใน Vec เดียว) ได้อย่างยืดหยุ่น
- 3.Type-Safe Error Handling:** ทุก Channel คืนค่า Result<(), String> (มีแค่ Ok หรือ Err) บังคับจัดการความผิดพลาดตั้งแต่ตอนเขียน ป้องกันข้อผิดพลาดหลุดรอดไป Runtime

## Challenge & Architectural Trade-offs

การวิเคราะห์ข้อแลกเปลี่ยนในทางวิศวกรรมซอฟต์แวร์ ระหว่างประสิทธิภาพความเร็วสูงสุดที่แลกมาด้วยการขยายตัวของขนาดไฟล์ไบนารี (Code Bloat จาก Monomorphization) กับความยืดหยุ่นในการจัดการชนิดข้อมูลขณะโปรแกรมทำงาน

### คำถาม : "หาก Static Dispatch เร็วสูงสุดระดับ Zero-Cost Abstraction และไร้ Runtime Overhead... ทำไม Rust ถึงไม่บังคับให้ใช้ Static Dispatch ในทุกสถานการณ์?"

#### การวิเคราะห์ข้อแลกเปลี่ยน (Trade-offs Analysis)

##### 1. ปัญหา Code Bloat & Memory Footprint:

การทำ Monomorphization จะก๊อปปี้โค้ด Machine Code แยกย่อยตามทุกชนิดข้อมูลจริง เช่น ถ้าฟังก์ชันหนึ่งถูกเรียกด้วย 20 Type คอมไพเลอร์จะสร้างฟังก์ชันซ้ำกัน 20 ชุดในไฟล์ Binary ส่งผลให้ขนาดไฟล์ .exe บวมขึ้นอย่างมหาศาล และทำให้ CPU เกิด Instruction Cache Miss (I-Cache Thrashing) เพราะคำสั่งเครื่องกระจัดกระจาย

##### 2. ข้อจำกัดด้าน Heterogeneous Collection & Extensibility:

Static Dispatch ไม่สามารถเก็บ Object ต่างชนิดกันไว้ใน List หรือ Vector เดียวกันได้

ไม่รองรับงานประเภท Plugin Architecture, GUI Frameworks (เช่น ปุ่ม, ช่องกรอก, เช็คบ็อกซ์ ในหน้าจอเดียวกัน) หรือระบบ Event Broadcaster แบบใน Exercise 2

#### สรุปกฎการตัดสินใจเลือกใช้ในงานจริง (Rule of Thumb)

##### • เลือกใช้ Static Dispatch เมื่อ:

- งานที่เน้น Critical Performance และความเร็วสูงสุด (เช่น Game Loop, DSP, Math Compute)
- กราฟชนิดข้อมูลที่แน่นอนล่วงหน้าตั้งแต่ตอนเขียนโค้ด

##### • เลือกใช้ Dynamic Dispatch เมื่อ:

- ต้องการรวม Object หลากหลายชนิดไว้ใน Collection เดียวกัน (Heterogeneous)
- ต้องการความยืดหยุ่น เช่น ทำ Plugin System, ไดรเวอร์อุปกรณ์, หรือโค้ดที่ต้องเชื่อมต่อกับบุคคลภายนอก (Third-party extensions)

# Key Takeaways — สรุปสิ่งที่ควรรจำ

1

Generics: เขียน Logic เดียว รองรับหลาย Type พร้อม Type Safety

2

Traits & Trait Bounds: กำหนด Shared Behavior เป็นสัญญา (Contract) บังคับความสามารถของ Type ป้องกันข้อผิดพลาดตั้งแต่ Compile Time

3

Monomorphization & Polymorphism: คอมไพเลอร์สร้าง Machine Code แยกเฉพาะ Type จริงช่วง Compile Time ได้ประสิทธิภาพระดับ Zero-Cost ต่างจาก Java (Type Erasure) หรือ Python (Duck Typing)

4

PPL Trade-offs: เลือกระหว่าง Static Dispatch (เร็วสูงสุด Zero-Cost แลกกับ Code Bloat) กับ Dynamic Dispatch (ยืดหยุ่นผ่าน Fat Pointer / Vtable แลกกับ Runtime Overhead)

# References — เอกสารอ้างอิง

- The Rust Programming Language (The Rust Book) — <https://doc.rust-lang.org/book/>
- The Rust Reference — <https://doc.rust-lang.org/reference/types/trait-object.html>
- Rust by Example — <https://doc.rust-lang.org/rust-by-example/>
- Concepts of Programming Languages (12th Edition)
- Rust RFC 0255 — Object Safety — <https://github.com/rust-lang/rfcs/blob/master/text/0255-object-safety.md>
- Python official docs — <https://typing.python.org/en/latest/reference/generics.html>
- cppreference — <https://cppreference.com/>
- Oracle Java Tutorials — <https://docs.oracle.com/javase/tutorial/>
- JetBrains : Rust vs Java — <https://blog.jetbrains.com/rust/2025/08/01/rust-vs-java/>

*GitHub Repository: <https://github.com/670710626/rust-tutorial-2569> (Pull Request: #43).*

# Questions & Answers

နံပါတ် 15 — Generics & Type System